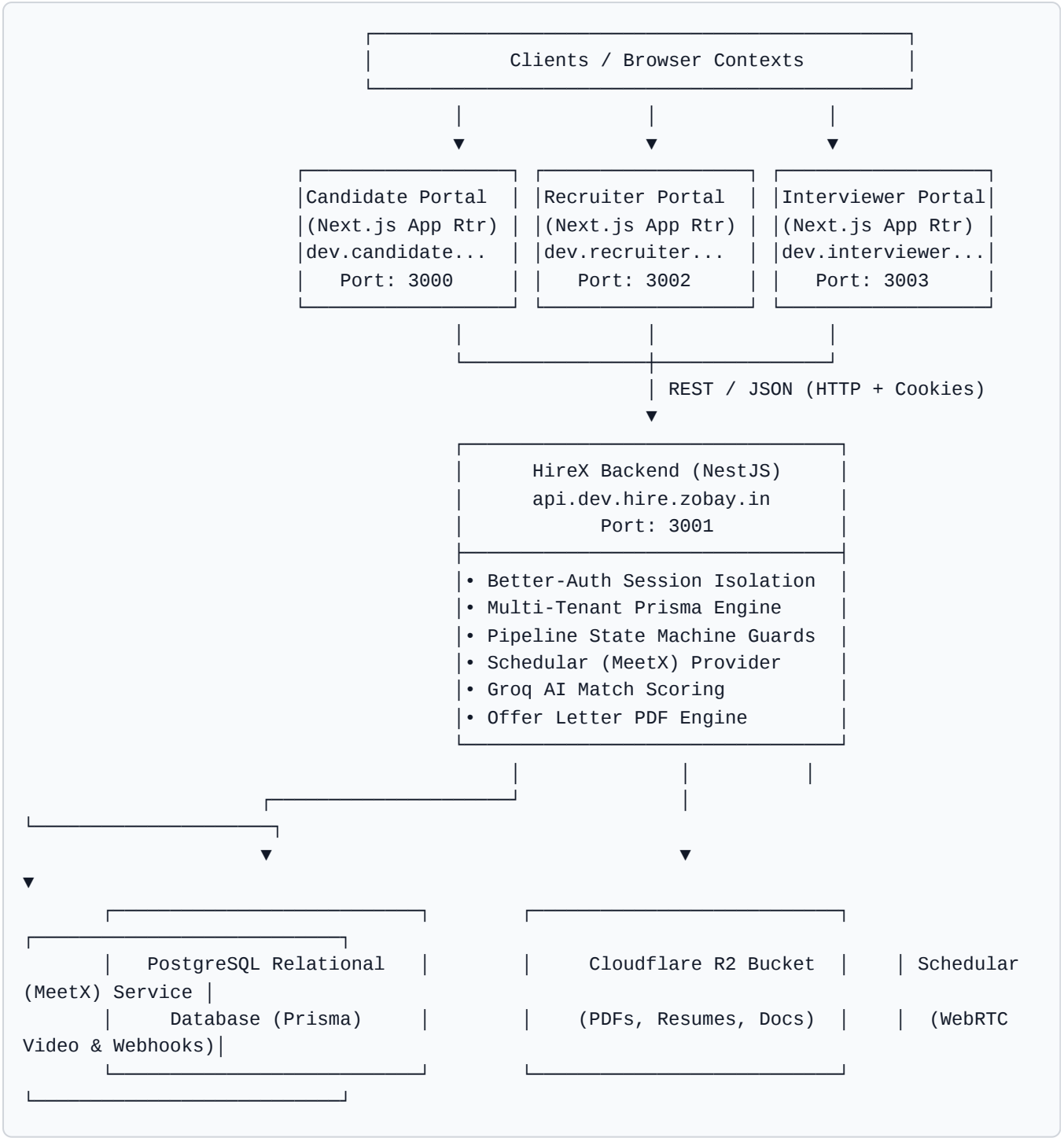
HireX — Technical Requirements Document (TRD)

Document ID: `HXP-HANDOVER-05`
Document Version: 1.0 (Production Architecture / Engineering Baseline)
Target Audience: Software Architects, Senior Developers, Tech Leads, DevOps, Security Reviewers
Last Updated: October 2026

---

1. System Architecture & Technology Stack

HireX is engineered as a decoupled, multi-tier micro-service system composed of one central API backend and three domain-specialized Next.js frontend portals.



Verified Technology Matrix

Layer	Technologies Used	Key Libraries & Tooling	Source Verification
Backend API	Node.js (v20+), NestJS (v10+), TypeScript	`@nestjs/core`, `@nestjs/platform-express`, `@nestjs/event-emitter`, `@nestjs/schedule`, `better-auth`, `pdfkit`, `xmlbuilder2`	`hirex-Backend/package.json`

Layer	Technologies Used	Key Libraries & Tooling	Source Verification
Database ORM	PostgreSQL 15+, Prisma ORM (v5+)	`@prisma/client`, `prisma`, connection pooling	`hirex-Backend/prisma/schema.prisma`
Recruiter Portal	Next.js 14/15 App Router, React 18/19	`tailwindcss`, `recharts`, `lucide-react`, `tailwind-merge`	`hirex-employer-frontend/package.json`
Candidate Portal	Next.js 14/15 App Router, React 18/19	`tailwindcss`, `lucide-react`, Custom Proxy (`src/proxy.ts`)	`hirex-frontend/package.json`
Interviewer Portal	Next.js 14/15 App Router, React 18/19	`tailwindcss`, `lucide-react`	`interviewer/package.json`
Object Storage	Cloudflare R2 (S3 API)	`@aws-sdk/client-s3`, `@aws-sdk/s3-request-presigner`	`hirex-Backend/src/documents/`
Video & Scheduling	Schedular / MeetX REST API & Webhooks	HMAC SHA-256 signature validation, WebSockets/WebRTC	`hirex-Backend/src/interviews/providers/`

---

2. Directory Structure & Application Entry Points

```
HireX/
├── hirex-Backend/                    # NestJS API Backend (Port: 3001)
│   ├── prisma/                       # Prisma Schema & Migrations
│   │   ├── schema.prisma             # 25 Models, 13 Enums
│   │   └── migrations/               # Historical schema migration directory
│   ├── src/
│   │   ├── main.ts                   # Application bootstrap, CORS, Cookie parser,
Global validation pipe
│   │   ├── app.module.ts             # Root application module aggregating 18 sub-
modules
│   │   ├── auth/                     # Better-Auth integration, Session isolation,
Guards, Password reset
│   │   ├── candidates/               # Candidate profile, auth, and application views
│   │   ├── recruiters/               # Recruiter company onboarding, pipeline, team RBAC
│   │   ├── interviewer/              # Interviewer portal auth and dashboard services
│   │   ├── interviews/               # Availability, Slots generation, Booking,
Schedular provider
│   │   ├── jobs/                     # Job requisitions, screening questions
│   │   ├── applications/             # Application lifecycle, state machine, AI match
scoring
│   │   ├── offers/                   # Branded PDF offer letter generation and
expiration cron
│   │   ├── documents/                # Multi-file compliance upload, verification,
Cloudflare R2
│   │   ├── search/                   # Role-scoped global search (/search?q=)
│   │   ├── webhooks/                 # Schedular (MeetX) HMAC-signed webhook handlers
│   │   └── common/                   # DTOs, interceptors, filters, and AI scorers
│   └── test/                         # 11 End-to-end Jest test suites
├── hirex-employer-frontend/          # Next.js Recruiter Portal (Port: 3002)
│   └── src/app/                      # App router: /applicants, /jobs, /interviewers,
/career-page, /reports
├── hirex-frontend/                   # Next.js Candidate Portal (Port: 3000)
│   ├── src/app/                      # App router: /jobs, /apply,
/candidate/applications, /interviews
│   └── src/proxy.ts                  # Reverse proxy detecting CNAME host headers and
rewriting to /careers/:slug
└── interviewer/                      # Next.js Interviewer Portal (Port: 3003)
    └── src/app/                      # App router: /dashboard, /availability,
/interviews/[id]/feedback
```

---

3. Database Architecture & Schema Specification

The relational schema is anchored on strict multi-tenancy. Every company-specific table includes `companyId`.

3.1 Schema Enums (13 Enums)

```
enum UserRole { CANDIDATE, RECRUITER, INTERVIEWER, ADMIN }
enum RecruiterRole { ADMIN, RECRUITER, HIRING_MANAGER }
enum RecruiterStatus { ACTIVE, INVITED, SUSPENDED }
enum JobStatus { DRAFT, OPEN, PAUSED, CLOSED }
enum SectionType { DEFAULT, CUSTOM }
enum QuestionType { SHORT_TEXT, LONG_TEXT, MULTIPLE_CHOICE, CHECKBOX, DROPDOWN, NUMBER,
DATE, FILE_UPLOAD }
enum ApplicationStatus { APPLIED, SCREENING, INTERVIEW, OFFER_STAGE,
DOCUMENT_VERIFICATION, HIRED, REJECTED, HOLD }
enum InterviewType { VIDEO, PHONE, IN_PERSON }
enum InterviewStatus { ASSIGNED, CONFIRMED, COMPLETED, CANCELLED }
enum SlotStatus { AVAILABLE, BOOKED, CANCELLED }
enum Recommendation { STRONG_HIRE, HIRE, LEANING_HIRE, NO_HIRE }
enum OfferStatus { DRAFT, EXTENDED, ACCEPTED, DECLINED, EXPIRED }
enum DocumentStatus { PENDING, VERIFIED, REJECTED }
```

3.2 Core Model Relationships

```
Company (Tenant Root)
  ├── 1..* Recruiter (userId, companyId, role)
  ├── 1..* Job (companyId, recruiterId, status)
  │      └── 1..1 ApplicationForm (JSONB schema snapshot)
  ├── 1..* Candidate (userId, companyId)
  │      └── 1..* Application (candidateId, jobId, companyId, status)
  │             ├── 0..* Interview (applicationId, status, meetingId, hostJoinUrl,
candidateJoinUrl)
  │             │      └── 0..1 InterviewFeedback (ratings JSONB, culturalFitScore,
recommendation)
  │             ├── 0..* Offer (applicationId, status, pdfUrl, expiresAt)
  │             └── 0..* Document (applicationId, type, status, rejectionReason,
verifiedAt, verifiedBy)
  │                    └── 1..* DocumentFile (fileName, fileUrl, fileKey, mimeType)
  └── 1..* Interviewer (userId, companyId, timezone, sharePhotoWithCandidates)
         └── 1..* InterviewAvailability (dayOfWeek, startTime, endTime, isRecurring)
```

---

4. Authentication, Session Security & ADR-001 Isolation

4.1 Better-Auth Architecture

HireX uses Better-Auth cryptographic primitives for user authentication. Sessions are tracked via HTTP-only cookie `better-auth.session_token`.

4.2 Portal Session Isolation (ADR-001)

To prevent cross-portal session contamination between Candidate, Recruiter, and Interviewer roles:

1. **Extraction:** The backend `session-cookie.ts` utility extracts the cookie carrier.

2. **Context Resolution:** The user record is fetched from PostgreSQL. The backend checks `user.role`:
   - `RECRUITER`: Permitted to access `/recruiters/*`, `/jobs/*`, `/reports/*`.
   - `INTERVIEWER`: Permitted to access `/interviewer/*`, `/interviews/*`.
   - `CANDIDATE`: Permitted to access `/candidate/*`, `/applications/*`.
3. **Role Guards:** If a Candidate session attempts to query Recruiter endpoints, the request is rejected with `403 Forbidden` and the user is redirected to the respective login screen.

---

5. API Endpoint Catalog (31 Controllers)

Method	Route	Controller	Auth Scope	Description
`POST`	`/api/auth/sign-in/email`	`AuthController`	Public	Standard credential login
`POST`	`/api/auth/sign-up/email`	`AuthController`	Public	Candidate registration
`POST`	`/recruiters/signup`	`RecruiterSignupController`	Public	Organization registration + recruiter admin seat
`POST`	`/auth/interviewer/login`	`InterviewerAuthController`	Public	Interviewer login with rememberMe and demo auto-seed
`GET`	`/auth/interviewer/invite`	`InterviewerAuthController`	Public	Validate single-use invite token
`POST`	`/auth/interviewer/invite/accept`	`InterviewerAuthController`	Public	Redeem token and activate interviewer password
`GET`	`/jobs`	`JobsController`	Public	Search and list active public job requisitions
`POST`	`/recruiters/jobs`	`JobsController`	Recruiter	Publish job with dynamic application form schema
`GET`	`/jobs/:id/application-form`	`ApplicationFormController`	Public	Fetch dynamic question schema for apply wizard
`POST`	`/applications`	`ApplicationsController`	Candidate	Submit application responses and resume snapshot

Method	Route	Controller	Auth Scope	Description
`POST`	`/public/guest-resume`	`ApplicationsController`	Public	Unauthenticated guest resume upload
`PATCH`	`/applications/:id/status`	`ApplicationsController`	Recruiter	Pipeline stage transition enforcing forward-only guards
`POST`	`/interviews/availability`	`AvailabilityController`	Interviewer	Save weekly recurring available hours
`POST`	`/interviews/slots/generate`	`SlotsController`	Recruiter	Compute candidate booking slots
`POST`	`/interviews/booking`	`BookingController`	Candidate	Atomically book slot and generate MeetX video room
`POST`	`/interviews/:id/feedback`	`InterviewsController`	Interviewer	Submit or update rubric scorecard + Cultural Fit score
`POST`	`/webhooks/schedular`	`SchedularWebhookController`	Webhook / HMAC	Handle `meeting.ended` and `meeting.mom.ready` events
`POST`	`/recruiters/offers/generate`	`OffersController`	Recruiter	Interpolate template placeholders and build offer PDF
`POST`	`/candidate/offers/:id/accept`	`OffersController`	Candidate	Sign offer; auto-shifts to `DOCUMENT_VERIFICATION`
`POST`	`/candidate/documents/upload`	`DocumentsController`	Candidate	Multi-file document upload to Cloudflare R2
`PATCH`	`/recruiters/documents/:id/verify`	`DocumentsController`	Recruiter	Verify candidate compliance document
`PATCH`	`/recruiters/documents/:id/reject`	`DocumentsController`	Recruiter	Reject document with mandatory reason string
`GET`	`/feeds/indeed-jobs.xml`	`FeedsController`	Public	XML feed conforming to Indeed aggregator specs
`GET`	`/feeds/linkedin-jobs.xml`	`FeedsController`	Public	XML feed conforming to LinkedIn aggregator specs

Method	Route	Controller	Auth Scope	Description
`GET`	`/feeds/naukri-jobs.xml`	`FeedsController`	Public	XML feed conforming to Naukri aggregator specs
`GET`	`/public/career-domain`	`CareerPageController`	Public	Resolves tenant slug for custom CNAME domain requests
`GET`	`/search`	`SearchController`	Authenticated	Role-scoped global search for `Ctrl+K` command palette

---

6. Schedular (MeetX) Video Integration & Webhook Lifecycle

6.1 Provider Pattern (`MeetingProvider`)

```
export interface MeetingProvider {
  createMeeting(options: MeetingOptions): Promise<MeetingResult>;
  getTranscript?(meetingId: string): Promise<any>;
  getMom?(meetingId: string): Promise<any>;
}
```

- **Fallback Dev Simulation:** When `SCHEDULAR_API_KEY` is empty, returns `https://meet.hirex.dev/join/${meetingId}` for safe offline testing.
- **Live Video Calling:** When configured, dispatches `POST https://api.dev.scheduler.zobay.in/api/v1/meetings`. It saves `hostJoinUrl` (privileged host controls) and `candidateJoinUrl` (attendee link).

6.2 HMAC-Signed Webhook Ingestion

- Inbound webhooks sent to `POST /webhooks/schedular` must supply header `x-signature`.
- The backend verifies HMAC SHA-256 against `process.env.SCHEDULAR_WEBHOOK_SECRET`.
- On `meeting.ended`, the interview status transitions to `COMPLETED`.
- On `meeting.mom.ready`, the system calls `getMom()` and `getTranscript()` to attach minutes of meeting to the interview record.

---

7. Cloudflare R2 Storage & File Management

- **Storage Provider:** Cloudflare R2 bucket (`hirex-test-dev`) configured via AWS S3 SDK.
- **Signed URLs:** Documents and resumes generate secure presigned URLs for in-browser PDF previews.

- **Cascade Safety:** Prisma schema enforces `onDelete: SetNull` on `Application.resume` so candidate deletion does not trigger unhandled foreign key crashes.

---

8. Scheduled Background Tasks

HireX implements `@nestjs/schedule` for asynchronous jobs:

- **Offer Expiration Cron (`@Cron('0 * * * *')`):** Runs every hour at minute 0.
  - Queries all `Offer` rows where `status == EXTENDED` and `expiresAt < NOW()`.
  - Atomically updates them to `EXPIRED`.
  - Updates application status to notify recruiter of stale offer.

---

9. Environment Variables Inventory (Names Only)

```
PORT
FRONTEND_URL
RECRUITER_URL
INTERVIEWER_URL
NODE_ENV
DATABASE_URL
BETTER_AUTH_SECRET
BETTER_AUTH_URL
COOKIE_DOMAIN
DEMO_AUTH_BYPASS
CLOUDFLARE_ACCOUNT_ID
CLOUDFLARE_R2_ACCESS_KEY_ID
CLOUDFLARE_R2_SECRET_ACCESS_KEY
CLOUDFLARE_R2_BUCKET
CLOUDFLARE_R2_PUBLIC_URL
ENCRYPTION_KEY
SCHEDULAR_API_BASE_URL
SCHEDULAR_API_KEY
SCHEDULAR_WEBHOOK_SECRET
SMTP_HOST
SMTP_PORT
SMTP_USER
SMTP_PASS
SMTP_FROM
GROQ_API_KEY
GROQ_MODEL
LINKEDIN_JOB_POSTING_URL
NAUKRI_JOB_POSTING_URL
INDEED_JOB_POSTING_URL
```

---

10. Local Development Setup & Verification Checklist

```
# 1. Database Setup
cd hirex-Backend
npx prisma migrate deploy
npm run seed  # Pre-seeds demo recruiter, interviewer, candidate, and company

# 2. Compile Check (0 Errors Required)
npx tsc --noEmit
cd ../hirex-employer-frontend && npx tsc --noEmit
cd ../hirex-frontend && npx tsc --noEmit
cd ../interviewer && npx tsc --noEmit

# 3. Automated Test Verification
cd ../hirex-Backend
npm test -- --runInBand  # 34 suites, 271 unit tests must pass
```